AI Product Intelligence System

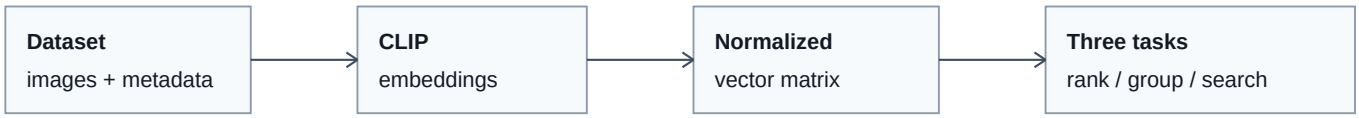
Day 2 Homework Report | Utkarsh Kumar Rai

1. Scope and shared approach

I implemented all three tasks in one notebook. There is no web interface; each task is a Python function followed by a table and product visualization.

The notebook tries the linked Kaggle dataset and CLIP first. If they are not available, it runs a small built-in demo so the complete logic can still be checked.

Shared pipeline



2. Task 1 - complementary recommendation

The fashion dataset does not include purchase history. I therefore use simple category rules as a practical first version: sports shoes map to socks, watches, bottles and backpacks. In production, these rules should be learned from real bought-together data.

Final selection: filter candidates to one complementary category, calculate 70% text-image relevance plus 30% visual style similarity, and keep the highest-scoring item. One result per category keeps the list varied.

Notebook result screenshot - input: Black Running Shoe A

Sports Socks	Fitness Watch	Water Bottle
Category: Socks	Category: Watches	Category: Bottle
Score: 0.911	Score: 0.911	Score: 0.789

Screenshots shown here are from the notebook demo mode; Kaggle mode runs the same selection logic with CLIP.

Duplicate Removal and Reverse Search

Day 2 Homework Report | Utkarsh Kumar Rai

3. Task 2 - unique product catalog

I calculate cosine similarity between product image embeddings. A pair is grouped only when its score is at least 0.95 and its product category also matches. The category check prevents a black shoe and a black watch from being treated as duplicates.

Connected duplicate pairs form a group. The clearest/largest image is kept; if image quality is equal, the lowest product ID is used so the final choice is repeatable.

Notebook result screenshot - duplicate grouping

Group 1

IDs: 1001,1002,1003

Keep: 1001

Group 2

IDs: 3001,3002

Keep: 3001

Final catalog

14 -> 11 products

F1: 1.00 demo

4. Task 3 - reverse text search

The function reverse_search converts a text query into a CLIP text embedding. It compares this vector with all image embeddings, sorts by cosine similarity, removes low-score results, and returns the top products.

Notebook result screenshot - query: "blue casual shirt"

Rank 1

Blue Casual Shirt A

Score: 1.000

Rank 2

Blue Casual Shirt B

Score: 1.000

Rank 3

Blue Casual Shirt C

Score: 1.000

Evaluation, Limitations and Running

Day 2 Homework Report | Utkarsh Kumar Rai

5. Evaluation

Recommendation	Complementary-category coverage@K and manual relevance review. Real purchase quality needs transaction data.
Duplicate catalog	Pairwise precision, recall and F1 on labeled pairs; otherwise manually review high-score pairs before fixing the threshold.
Reverse search	Recall@3, Recall@5, Recall@10 and Mean Reciprocal Rank (MRR) on queries with known relevant product IDs.
Failure checks	Unclear images, uncommon products, wrong colors, and searches whose best score is below the confidence threshold.

Demo search metrics

4 test queries: Recall@3 = 1.00 | Recall@5 = 1.00 | Recall@10 = 1.00 | MRR = 1.00

These scores validate the demo pipeline only; they are not claimed as final real-world performance.

6. Limitations and practical next steps

- The demo uses a small controlled catalog; real CLIP results must be checked on the linked dataset.
- The duplicate threshold should be tuned using manually labeled duplicate/non-duplicate pairs.
- Recommendations should later use real co-purchase logs rather than only category rules.
- For a full dataset, use a vector index such as FAISS instead of comparing every pair.

7. How to run the notebook

1. Upload the notebook to Kaggle and attach the Fashion Product Images Small dataset.
2. Enable a GPU and Internet once for openai/clip-vit-base-patch32, or attach the model files.
3. Set RUN_MODE to "kaggle" and run all cells. Result CSV files are saved in /kaggle/working.
4. For a quick check without the dataset, set RUN_MODE to "demo".